MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 6 | FRONTEND | REACT.JS | ROMAN URDU

CHAPTER 6

React.js Basics — Frontend Shuru Karo!

Components, JSX, Props, useState, useEffect — React ki duniya mein khush aamdeed!

Is Chapter Mein Aap Seekhenge:

- React kya hai — library vs framework ka farq
- Create React App se project setup karna
- JSX — JavaScript mein HTML likhna
- Components — reusable UI building blocks banana
- Props — parent se child ko data bhejna
- useState Hook — component ka apna data manage karna
- useEffect Hook — side effects aur API calls
- CSS Modules aur Styling approaches
- ShopEase frontend folder structure

■ Axios se Backend API ko connect karna

■ Backend Complete!	Chapters 1-5 mein Express, MongoDB, MVC, aur Auth complete hua. Chapter 6 se hum React ke saath ShopEase ka chehra banayenge!

TABLE OF CONTENTS

7.0 useEffect Hook

- 7.1 Side effects kya hain?
- 7.2 useEffect syntax
- 7.3 Dependency array
- 7.4 Cleanup function
- 7.5 API call karna

8.0 Styling in React

- 8.1 Inline styles
- 8.2 CSS Modules
- 8.3 className vs class
- 8.4 Conditional styling

9.0 ShopEase Frontend Setup

- 9.1 Folder structure
- 9.2 Axios install aur setup
- 9.3 API service file banana
- 9.4 Pehla component — ProductCard

10.0 Summary & Practice

- 10.1 Chapter recap
- 10.2 Practice exercises
- 10.3 Chapter 7 preview

SECTION 1 — React Kya Hai?

1.0 — React Kya Hai?

React ek **JavaScript library** hai jo Facebook ne 2013 mein banaya. Iska ek hi kaam hai — **user interface banana**. Jaise LEGO blocks hote hain — chhote chhote pieces banao aur milake ek poori website bana lo. React mein yeh pieces **Components** kehlaate hain.

1.1 — Library vs Framework — Farq Kya Hai?

	Library (React)	Framework (Angular, Vue)
Control	Tum decide karo — kya use karna	Framework decide karta hai — sab kuch
Learning	Ek cheez seekho — UI	Poora ecosystem seekhna padta hai
Flexibility	Bohat zyada — mix karo kuch bhi	Kam — framework ka tarika follow karo
Size	Chhota — sirf UI library	Bada — poora framework
Routing	React Router alag install	Built-in routing
State Mgmt	Redux/Zustand alag install	Often built-in
Use Case	UI components — MERN stack	Enterprise apps — full structure

1.2 — Virtual DOM — React Ka Secret Weapon

Browser ka **Real DOM** update karna slow hota hai. React ne ek **Virtual DOM** banaya — RAM mein ek copy. Jab kuch change hota hai, React pehle Virtual DOM update karta hai, phir Real DOM se compare karta hai, aur sirf jo badla woh update karta hai. Yeh bahut fast hota hai!

■ Real DOM (Browser)	■ Virtual DOM (React)
Kuch bhi change karo — poora page re-render	Sirf jo badla woh update — baaki same rehta
Slow — specially bade pages pe	Fast — minimal DOM operations
Jaise puri kitaab reprint karo ek word ke liye	*Jaise sirf woh page reprint karo jahan change hua*

■ **TIP**

React 'Reconciliation' process se Virtual DOM aur Real DOM ko sync karta hai — is process mein sirf changed nodes update hote hain. Isliye React itna fast hai!

SECTION 2 — Project Setup

2.0 — Create React App

ShopEase ke backend folder ke **barabar** ek alag frontend folder banao. Dono alag projects hain — alag terminals mein chalenge:

```
# Pehle shopease folder ke andar jao (backend wala)
cd ~/Desktop/shopease

# React app banao — 'frontend' naam se
npx create-react-app frontend

# Ya Vite use karo (faster — recommended!)
npm create vite@latest frontend -- --template react

# Frontend folder mein jao
cd frontend

# Dependencies install karo
npm install

# Development server start karo
npm start   # CRA ke liye
npm run dev # Vite ke liye

# Browser mein khulega: http://localhost:3000
```

■ TIP

Ab 2 terminals chalenge: ek backend ke liye (port 5000), ek frontend ke liye (port 3000). VS Code mein Split Terminal use karo — bahut convenient hota hai!

2.1 — Default Folder Structure

```
frontend/
  ■■■ public/
        ■ ■■■ index.html
  ■■■ src/
        ■ ■■■ App.js
```

```
        ■ ■■■ App.css
        ■ ■■■ index.js
        ■ ■■■ index.css
    ■■■ package.json
    ■■■ .gitignore
```

File	Kaam
public/index.html	Ek hi HTML file — React yahan inject hota hai. Kabhi seedha mat badlo.
src/index.js	Entry point — ReactDOM.render() yahan hota hai. React app yahan start hoti hai.
src/App.js	Root component — poori app ka baap. Yahan se shuru karo.
src/App.css	App component ki CSS. Baad mein hum CSS Modules use karenge.

2.2 — src/index.js — React Ka Entry Point

```
// src/index.js — yahan se React start hoti hai
import React    from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App      from './App';

// public/index.html mein <div id='root'> hai
// React apna poora UI ussi div mein inject karta hai
const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// React.StrictMode development mein extra warnings dikhata hai
// Production build mein automatically remove ho jaata hai
```

SECTION 3 — JSX: JavaScript + HTML

3.0 — JSX Kya Hai?

JSX ek special syntax hai jo React mein use hota hai. Yeh JavaScript ke andar HTML jaisi language likhne deta hai. Browser directly JSX nahi samajhta — Babel tool ise regular JavaScript mein convert karta hai:

```
// JSX — jo hum likhte hain
const element = <h1 className='title'>Hello ShopEase!</h1>;

// Babel ise yeh JavaScript mein convert karta hai:
const element = React.createElement('h1', { className: 'title' }, 'Hello ShopEase!');

// JSX waala version zyada readable hai — isliye use karte hain
```

3.1 — JSX Ke Zaroori Rules

Rule	Galat ■	Sahi ■
Single root element	`<h1>Hi</h1><p>World</p>`	`<div><h1>Hi</h1><p>World</p></div>`
class ki jagah className	`<div class='box'>`	`<div className='box'>`
for ki jagah htmlFor	`<label for='name'>`	`<label htmlFor='name'>`
Self-closing tags	`  ya <img>`	`  ya <img />`
JavaScript mein {}	`<p>2+2</p>`	`<p>{2+2}</p>  (shows 4)`
camelCase attributes	`<div onclick='fn'>`	`<div onClick={fn}>`
Comments	`<!-- comment -->`	`{/* yeh JSX comment hai */}`

3.2 — JSX Mein JavaScript Expressions

Curly braces **{ }** ke andar koi bhi JavaScript expression likh sakte ho:

```
const productName = 'Nike Air Max';
const price       = 15000;
const isAvailable = true;
const images      = ['img1.jpg', 'img2.jpg'];
```

```
return (
  <div>
    {/* Variable use karna */}
    <h2>{productName}</h2>

    {/* Expression calculate karna */}
    <p>Price: Rs. {price.toLocaleString()}</p>

    {/* Ternary operator — conditional render */}
    <span>{isAvailable ? '■ In Stock' : '■ Out of Stock'}</span>

    {/* && operator — only show if true */}
    {isAvailable && <button>Add to Cart</button>}

    {/* Array map — list render karna */}
    <ul>
      {images.map((img, index) => (
        <li key={index}>{img}</li>
      ))}
    </ul>
  </div>
);
```

■ YAAD RAHO

JSX mein if/else directly nahi likh sakte — sirf expressions chalte hain. Ternary (? :) ya && operator use karo conditional rendering ke liye. Ya phir if/else JSX se bahar likhke variable return karo.

SECTION 4 — Components

4.0 — Component Kya Hai?

Component ek **reusable UI piece** hai — jaise LEGO block. Amazon pe har product ka card same dikhta hai — ek Component banao, baar baar use karo. Component ek JavaScript function hai jo JSX return karta hai:

4.1 — Pehla Component Banana

```
// src/components/Greeting.jsx

// Component ek function hai — naam capital letter se shuru hota hai
function Greeting() {
  return (
    <div>
      <h1>Assalam o Alaikum!</h1>
      <p>ShopEase mein khush aamdeed!</p>
    </div>
  );
}

// Export karna zaroori hai — doosri jagah use ho sake
export default Greeting;
```

```
// src/App.js — Component use karna
import Greeting from './components/Greeting';

function App() {
  return (
    <div className='App'>
      {/* Component ek HTML tag ki tarah use hota hai */}
      <Greeting />
      <Greeting />  {/* Same component do baar! */}
    </div>
  );
}
```

```
export default App;
```

4.2 — Component Ke Rules

Capital Letter	Component ka naam HAMESHA capital letter se shuru hona chahiye — Greeting, ProductCard, Navbar. Lowercase = HTML tag samjha jaata hai.
JSX return karo	Component ek JSX return karta hai — yahi screen pe dikhta hai. Ek single root element mein wrap karo.
export default	Component ko export karo taki doosri files import kar sakein. Ek file mein ek default export hoti hai.
import karna	Jahan use karna ho wahan import karo. Path relative hota hai — ./components/Greeting
Pure function	Component ko side effects nahi karne chahiye return ke dauran. useEffect ke liye alag section hai.

4.3 — ShopEase Ke Components Ka Plan

Component	File	Kya Dikhata Hai?
Navbar	components/Navbar.jsx	Top navigation bar — logo, search, cart icon, login
Footer	components/Footer.jsx	Footer — links, copyright
ProductCard	components/ProductCard.jsx	Ek product ka card — image, naam, price, rating
Loader	components/Loader.jsx	Loading spinner — API call ke dauran
Message	components/Message.jsx	Error ya success message dikhana
Rating	components/Rating.jsx	Star rating display — 1 se 5
Paginate	components/Paginate.jsx	Pagination buttons
SearchBox	components/SearchBox.jsx	Search input aur button
HomeScreen	screens/HomeScreen.jsx	Homepage — products grid
ProductScreen	screens/ProductScreen.jsx	Product detail page
CartScreen	screens/CartScreen.jsx	Shopping cart page

LoginScreen	`screens/LoginScreen.jsx`	Login form
RegisterScreen	`screens/RegisterScreen.jsx`	Register form

SECTION 5 — Props

5.0 — Props Kya Hain?

Props (Properties) se hum parent component se child component ko data bhejte hain. Jaise function ke arguments hote hain — waise hi component ke props. Props **read-only** hain — child unhe change nahi kar sakta:

```
// ■■ Parent Component ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Props HTML attributes ki tarah pass karte hain
function App() {
  return (
    <div>
      <ProductCard
        name='Nike Air Max'
        price={15000}
        image='/images/nike.jpg'
        rating={4.5}
        inStock={true}
      />
      <ProductCard
        name='Levi Jeans'
        price={3500}
        image='/images/levi.jpg'
        rating={4.2}
        inStock={false}
      />
    </div>
  );
}

// ■■ Child Component ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Props ek object mein milte hain
function ProductCard({ name, price, image, rating, inStock }) {
  // Destructuring se directly variables mein
```

```
  return (
    <div className='product-card'>
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>Rs. {price.toLocaleString()}</p>
      <p>Rating: {rating} ■</p>
      {inStock
        ? <button>Add to Cart</button>
        : <p className='out-stock'>Out of Stock</p>
      }
    </div>
  );
}
```

5.1 — Default Props

```
// Agar prop pass na ho toh default value use karo
function ProductCard({ name, price, image = '/default.jpg', rating = 0 })
 {
  return (
    <div>
      <img src={image} alt={name} />  {/* default.jpg if not passed */}
      <p>Rating: {rating}</p>          {/* 0 if not passed */}
    </div>
  );
}

// Ya defaultProps use karo (purana tarika):
ProductCard.defaultProps = {
  image:  '/default.jpg',
  rating: 0,
};
```

5.2 — Children Prop — Special Prop

children ek special prop hai — opening aur closing tags ke beech jo likho woh milta hai:

```
// Card component — wrapper
```

```
function Card({ children, title }) {
  return (
    <div className='card'>
      <h2>{title}</h2>
      <div className='card-body'>
        {children}  {/* Jo andar rakha woh yahan aayega */}
      </div>
    </div>
  );
}

// Use karna:
<Card title='Featured Products'>
  <ProductCard name='Nike' price={5000} />
  <ProductCard name='Adidas' price={4500} />
  <p>Yeh sab children hain!</p>
</Card>
```

TIP

Props sirf parent se child ko jaate hain — yeh ek direction hai. Child se parent ko data bhejna ho toh callback function prop pass karo — ise 'lifting state up' kehte hain. Chapter 7 mein dekhenge!

SECTION 6 — useState Hook

6.0 — State Kya Hai?

State component ka **apna data** hai jo time ke saath change ho sakta hai. Jab state change hoti hai, React automatically component ko re-render karta hai — screen update ho jaati hai. Props bahar se aata hai, State andar se hota hai:

Props	State
Parent se milta hai	Component khud manage karta hai
Read-only — change nahi kar sakte	Mutable — setState se change karo
Example: product ka naam	*Example: cart mein kitne items*

6.1 — useState Syntax

```
import { useState } from 'react';

function Counter() {
  // useState(initialValue) — array return karta hai:
  // [currentValue, setterFunction]
  const [count, setCount] = useState(0);
  //     ↑ value  ↑ setter  ↑ initial value

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

// Har baar button click hoga — setCount chalega
// State update hogi — component re-render hoga
```

```
// Screen pe naya count dikh jaayega
```

6.2 — Multiple States aur Alag Types

```
function ProductScreen() {
  // Alag alag types ki state
  const [product,   setProduct]   = useState(null);     // Object
  const [loading,   setLoading]   = useState(true);     // Boolean
  const [error,     setError]     = useState('');       // String
  const [quantity,  setQuantity]  = useState(1);        // Number
  const [reviews,   setReviews]   = useState([]);       // Array

  return (
    <div>
      {loading && <p>Loading...</p>}
      {error  && <p className='error'>{error}</p>}
      {product && (
        <div>
          <h1>{product.name}</h1>
          <p>Qty: {quantity}</p>
          <button onClick={() => setQuantity(q => q + 1)}>+</button>
          <button onClick={() => setQuantity(q => Math.max(1, q - 1))}>-<
/button>
        </div>
      )}
    </div>
  );
}
```

6.3 — Form State — Complete Example

```
function LoginForm() {
  const [email,    setEmail]    = useState('');
  const [password, setPassword] = useState('');
  const [loading,  setLoading]  = useState(false);
  const [error,    setError]    = useState('');

  const handleSubmit = async (e) => {
```

```
    e.preventDefault();  // Page reload rokna
    setLoading(true);
    setError('');

    try {
      // API call (Section 9 mein dekhenge)
      console.log('Login:', email, password);
      setLoading(false);
    } catch (err) {
      setError(err.message);
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      {error && <p className='error'>{error}</p>}

      <input
        type='email'
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder='Email'
      />
      <input
        type='password'
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder='Password'
      />
      <button type='submit' disabled={loading}>
        {loading ? 'Logging in...' : 'Login'}
      </button>
    </form>
  );
}
```

■ **TIP**

Controlled Components — input ki value hamesha state se aati hai (value={email}), aur onChange se state update hoti hai. Yeh React ka standard pattern hai — form ko React control karta hai, browser nahi.

SECTION 7 — useEffect Hook

7.0 — Side Effects Kya Hain?

Side effects woh kaam hain jo component ke render se bahar hote hain — API calls, timers, event listeners, localStorage, etc. **useEffect** inhe handle karne ke liye hai:

Side Effect	Example
API call	Backend se products fetch karna
Document title	Page title 'Nike Shoes | ShopEase' set karna
Timer	Auto-logout 30 min baad
Event listener	Window resize detect karna
localStorage	Token save/load karna
Subscription	WebSocket se live updates

7.1 — useEffect Syntax aur Dependency Array

```
import { useState, useEffect } from 'react';

// ■■ Form 1: Dependency array nahi — HAR render ke baad chale ■■
useEffect(() => {
  console.log('Har render ke baad chalta hai!');
});  // ← koi array nahi

// ■■ Form 2: Empty array [] — sirf PEHLI baar chale ■■■■■■■■■■■■■■
useEffect(() => {
  console.log('Sirf component mount hone pe chalta hai!');
  // API call, initial data fetch yahan karo
}, []);  // ← empty array

// ■■ Form 3: Dependencies — jab woh value change ho tab chale ■■■
useEffect(() => {
  console.log('productId change hua:', productId);
  // Naye productId ke liye data fetch karo
}, [productId]);  // ← jab productId badlega tab chalega
```

```
// ■■ Form 4: Cleanup function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
useEffect(() => {
  const timer = setInterval(() => console.log('tick'), 1000);

  // Cleanup: component unmount hone pe yeh chalega
  return () => {
    clearInterval(timer);  // Timer band karo — memory leak rokna
  };
}, []);
```

7.2 — API Call with useEffect — Complete Pattern

```
function HomeScreen() {
  const [products, setProducts] = useState([]);
  const [loading,  setLoading]  = useState(true);
  const [error,    setError]    = useState('');

  useEffect(() => {
    // Async function useEffect mein directly nahi — alag banao
    const fetchProducts = async () => {
      try {
        setLoading(true);
        const response = await fetch('http://localhost:5000/api/products');
        const data     = await response.json();

        if (data.success) {
          setProducts(data.data);
        } else {
          setError(data.message);
        }
      } catch (err) {
        setError('Server se connect nahi ho saka!');
      } finally {
        setLoading(false);  // Success ya error — loading band karo
      }
```

```
    };

    fetchProducts();
  }, []);  // Sirf ek baar — component mount pe

  if (loading) return <div>Loading products...</div>;
  if (error)   return <div className='error'>{error}</div>;

  return (
    <div className='products-grid'>
      {products.map(product => (
        <ProductCard
          key={product._id}  {/* MongoDB ka _id */}
          product={product}
        />
      ))}
    </div>
  );
}
```

■ YAAD RAHO

useEffect mein async function directly nahi likha jaata. Andar ek async function banao aur call karo — jaise fetchProducts() upar example mein. Yeh React ka required pattern hai!

SECTION 8 — Styling in React

8.0 — React Mein CSS Kaise Lagate Hain?

Method	Kaise?	Kab Use Karo?
External CSS	className='box' + App.css	Simple small projects
CSS Modules	styles.box — scoped CSS	Medium projects — ShopEase mein
Inline Styles	style={{ color: 'red' }}	Dynamic styles — conditional
Tailwind CSS	className='bg-blue-500 p-4'	Large projects — utility first
Styled Components	const Box = styled.div``	Component libraries

8.1 — External CSS (Sabse Asaan)

```
/* src/components/ProductCard.css */
.product-card {
  background: #161b22;
  border: 1px solid #30363d;
  border-radius: 8px;
  padding: 16px;
  transition: transform 0.2s;
}
.product-card:hover { transform: translateY(-4px); }
.product-card img   { width: 100%; border-radius: 4px; }
.product-price      { color: #56cf8a; font-size: 1.2rem; font-weight: bol
d; }
```

```
// ProductCard.jsx
import './ProductCard.css';  // CSS import karo

function ProductCard({ product }) {
  return (
    <div className='product-card'>  {/* class nahi — className! */}
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
```

```
      <p className='product-price'>Rs. {product.price}</p>
    </div>
  );
}
```

8.2 — Inline Styles — Dynamic Styling

```
// Inline styles — object notation — camelCase properties
function Badge({ type }) {
  const colors = {
    success: '#56CF8A',
    error:   '#FF6B6B',
    warning: '#FFD93D',
  };

  return (
    <span style={{
      backgroundColor: colors[type] || '#8B949E',
      color:           '#0D1117',
      padding:         '4px 12px',
      borderRadius:    '20px',
      fontWeight:      'bold',
      fontSize:        '12px',
    }}>
      {type.toUpperCase()}
    </span>
  );
}

// Use: <Badge type='success' /> ya <Badge type='error' />
```

■ TIP

Inline styles mein double curly braces {{ }} hote hain — bahari {} JSX expression hai, andar wala {} JavaScript object hai. CSS properties camelCase mein likhte hain: background-color = backgroundColor.

SECTION 9 — ShopEase Frontend Setup

9.0 — Final Folder Structure

```
frontend/src/
├── components/
│   ├── Navbar.jsx
│   ├── Footer.jsx
│   ├── ProductCard.jsx
│   ├── Loader.jsx
│   ├── Message.jsx
│   ├── Rating.jsx
│   └── Paginate.jsx
├── screens/
│   ├── HomeScreen.jsx
│   ├── ProductScreen.jsx
│   ├── CartScreen.jsx
│   ├── LoginScreen.jsx
│   ├── RegisterScreen.jsx
│   └── ProfileScreen.jsx
├── services/
│   └── api.js
├── utils/
│   └── helpers.js
├── App.js
├── App.css
└── index.js
```

9.1 — Axios Install aur API Service

```
# Axios install karo — fetch se better
npm install axios
```

■ src/services/api.js

```
import axios from 'axios';

// Axios instance banao — base URL set karo
const API = axios.create({
  baseURL: 'http://localhost:5000/api',
});

// Request interceptor — har request mein token add karo
API.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// ■■ Product API calls ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const getProducts = (keyword = '', page = 1) =>
  API.get(`/products?keyword=${keyword}&page=${page}`);

export const getProductById = (id) =>
  API.get(`/products/${id}`);

export const createProduct = (productData) =>
  API.post('/products', productData);

// ■■ User API calls ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const loginUser = (email, password) =>
  API.post('/users/login', { email, password });

export const registerUser = (name, email, password) =>
  API.post('/users/register', { name, email, password });

export const getUserProfile = () =>
  API.get('/users/profile');

// ■■ Order API calls ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const createOrder = (orderData) =>
  API.post('/orders', orderData);
```

```
export const getMyOrders = () =>
  API.get('/orders/myorders');
```

9.2 — ProductCard Component — Poora Code

■ src/components/ProductCard.jsx

```
import { Link } from 'react-router-dom';  // Chapter 7 mein install karen
ge

function ProductCard({ product }) {
  const { _id, name, image, price, rating, numReviews, stock } = product;

  return (
    <div style={{
      background:   '#161B22',
      border:       '1px solid #30363D',
      borderRadius: '12px',
      overflow:     'hidden',
      transition:   'transform 0.2s, box-shadow 0.2s',
    }}>
      {/* Product Image */}
      <img
        src={image || '/images/default.jpg'}
        alt={name}
        style={{ width: '100%', height: '200px', objectFit: 'cover' }}
      />

      <div style={{ padding: '16px' }}>
        {/* Product Name */}
        <h3 style={{ color: '#E6EDF3', fontSize: '14px', marginBottom: '8
px' }}>
          {name}
        </h3>

        {/* Rating */}
        <p style={{ color: '#FFD93D', fontSize: '13px' }}>
          {'■'.repeat(Math.round(rating))} ({numReviews} reviews)
```

```
        </p>

        {/* Price */}
        <p style={{ color: '#56CF8A', fontSize: '18px', fontWeight: 'bold
' }}>
          Rs. {price?.toLocaleString()}
        </p>

        {/* Stock Status */}
        <p style={{ color: stock > 0 ? '#56CF8A' : '#FF6B6B', fontSize: '
12px' }}>
          {stock > 0 ? `In Stock (${stock})` : 'Out of Stock'}
        </p>
      </div>
    </div>
  );
}

export default ProductCard;
```

9.3 — HomeScreen — Products Fetch aur Display

■ src/screens/HomeScreen.jsx

```
import { useState, useEffect } from 'react';
import ProductCard from '../components/ProductCard';
import { getProducts } from '../services/api';

function HomeScreen() {
  const [products, setProducts] = useState([]);
  const [loading,  setLoading]  = useState(true);
  const [error,    setError]    = useState('');

  useEffect(() => {
    const fetchProducts = async () => {
      try {
        const { data } = await getProducts();
        setProducts(data.data);
      } catch (err) {
```

```
        setError(err.response?.data?.message || 'Kuch galat ho gaya!');
      } finally {
        setLoading(false);
      }
    };
    fetchProducts();
  }, []);

  if (loading) return <p style={{color:'#61DAFB',textAlign:'center'}}>Loading...</p>;
  if (error)   return <p style={{color:'#FF6B6B',textAlign:'center'}}>{error}</p>;

  return (
    <div>
      <h1 style={{ color: '#E6EDF3', marginBottom: '24px' }}>Latest Products</h1>
      <div style={{
        display:             'grid',
        gridTemplateColumns: 'repeat(auto-fill, minmax(220px, 1fr))',
        gap:                 '20px',
      }}>
        {products.map(product => (
          <ProductCard key={product._id} product={product} />
        ))}
      </div>
    </div>
  );
}

export default HomeScreen;
```

SECTION 10 — Summary, Practice & Next Steps

10.0 — Chapter 6 Ka Recap

React	UI library — components se website banao
Virtual DOM	Fast rendering — sirf changed parts update karo
JSX	JS mein HTML likhna — Babel convert karta hai
JSX Rules	className, htmlFor, self-close, { } expressions
Components	Reusable UI pieces — capital name, JSX return, export
Props	Parent se child ko data — read-only, destructure karo
children prop	Opening/closing tags ke beech ka content
useState	Component ka apna data — [value, setter] = useState(init)
Controlled Input	value={state} + onChange={setter} — React controls form
useEffect	Side effects — API calls, timers — dependency array zaroori
useEffect forms	No array=every render | []=once | [x]=when x changes
Styling	className + CSS file, ya inline style={{ camelCase: val }}
Axios	API calls ke liye — interceptors se token auto-add
api.js	Centralized API service — saare calls ek jagah
ProductCard	Reusable product component — props se data receive
HomeScreen	useEffect + useState + map = products grid

10.1 — Practice Exercises

Easy Level

1. Create React App ya Vite se project banao
2. Greeting component banao — apna naam aur shehar dikhao
3. 2-3 ProductCard components render karo hardcoded data se
4. useState se ek counter banao — increment, decrement, reset

Medium Level

1. Props wala ProductCard banao — parent se data bhejo
2. useEffect se backend ke products fetch karo (server chal raha ho)
3. Loading aur error state properly handle karo
4. Login form banao — controlled inputs, state management

Challenge Level

1. HomeScreen banao — grid layout mein products
2. ProductCard pe hover effect add karo inline styles se
3. Search input banao — keyword state, useEffect dependency mein
4. Axios api.js file banao — interceptor se token attach karo
5. Loader component banao — loading state pe dikhao

10.2 — Chapter 7 Ka Preview

React Router & Advanced Hooks

- React Router v6 — install aur setup
- Routes aur Link — pages banana aur navigate karna
- useParams — URL se ID lena (:productId)
- useNavigate — programmatically page change karna
- useContext Hook — global state bina Redux ke
- useReducer Hook — complex state management
- Custom Hooks banana — reusable logic
- ShopEase ke saare pages connect karna

Frontend Shuru Ho Gaya! ■■■

Aaj tumne React ki duniya mein qadam rakha — components, JSX, props, useState, useEffect — yeh sab React ki reedh ki haddi hain. Har cheez ek ek karke aayegi. Practice karo, galtiyan karo, seekho. **Chapter 7 mein React Router se ShopEase ke pages banana shuru karenge!**